SENA 🟢

SERVICIO NACIONAL DE APRENDIZAJE

Centro de Teleinformática y Producción Industrial (CTPI) - Regional Cauca

EVIDENCIA DE CONOCIMIENTO: GA7-220501096-AA4-EV01

Taller Técnico: Componentes Front-end con React y JSX

Estudio conceptual y metodológico sobre los pilares del desarrollo frontend moderno con **React-JS**: análisis diferencial de **JSX** y transpilación con Babel, componentes de clase frente a funciones con **Hooks**, gestión de **SyntheticEvents** y mapa conceptual de arquitectura.

Aprendiz: Karen Yuliana Campo

Programa: Análisis y Desarrollo de Software (ADSO)

Ficha de Caracterización: 228118

Modalidad: 100% Virtual

Fase Formativa: Ejecución (AP7)

Actividad: GA7-220501096-AA4

Ciudad / Sede: Popayán, Cauca

Fecha de Entrega: Noviembre de 2026

1. Introducción y Objetivos

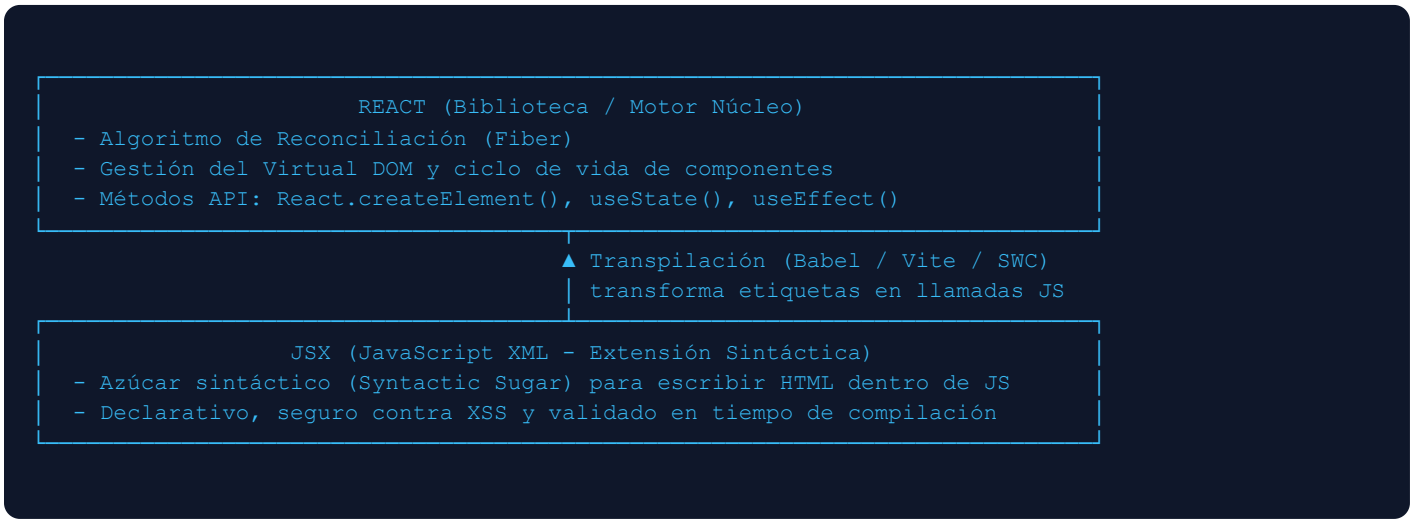
En la ingeniería de software actual, el desarrollo de interfaces de usuario (Frontend) ha experimentado una evolución paradigmática: se abandonó la manipulación directa e imperativa del Document Object Model (DOM) mediante JavaScript puro o bibliotecas tradicionales, dando paso a arquitecturas declarativas basadas en componentes reutilizables y reactivos.

En este contexto, **React** —desarrollada originalmente por Meta (Facebook) y respaldada por una vasta comunidad global de código abierto— se ha consolidado como la tecnología dominante para la construcción de Single Page Applications (SPA), interfaces web dinámicas y aplicaciones móviles multiplataforma.

> 🎯 **Objetivos del Taller Formativo:**
>
> - Analizar las diferencias técnicas, conceptuales y de procesamiento entre la biblioteca React y la extensión sintáctica JSX.
> - Examinar el concepto de clases en React, su ciclo de vida y contrastarlo con la evolución hacia componentes funcionales impulsada por los Hooks.
> - Describir el funcionamiento del sistema de eventos en React, su sintaxis camelCase, la delegación de eventos y el objeto unificado SyntheticEvent.
> - Diseñar un mapa conceptual representativo de la arquitectura de React, explicando el funcionamiento del Virtual DOM y la reconciliación.
> - Sintetizar las características clave que posicionan a React como el framework líder en la industria de desarrollo de software.

2. Diferencia entre React y JSX

Uno de los errores más comunes al iniciar en el ecosistema es confundir **React** con **JSX**, considerándolos términos intercambiables. En realidad, corresponden a dos conceptos distintos y complementarios:



React es la biblioteca responsable de orquestar el árbol de componentes, evaluar mutaciones de estado y sincronizar eficientemente la memoria con la pantalla.

JSX (JavaScript XML) es una extensión de la sintaxis estándar de ECMAScript. Permite estructurar la interfaz de forma declarativa y visualmente intuitiva. **JSX no es interpretado por el navegador**: herramientas como **Babel** o **SWC** lo transforman en llamadas nativas a React.createElement().

Demostración Técnica: Código JSX vs. Transpilación a JavaScript Puro

```
// 1. Código en JSX (escrito por el desarrollador en la Pizzería Bella Italia)
const TarjetaPizza = ({ nombre, precio }) => (
  <div className="card-pizza">
    <h3>{nombre}</h3>
    <p>${precio} COP</p>
    <button onClick={() => pedir(nombre)}>Ordenar</button>
  </div>
);

// 2. Transpilación real generada por Babel (ejecutada en el navegador)
const TarjetaPizza = ({ nombre, precio }) => {
  return React.createElement(
    "div",
    { className: "card-pizza" },
    React.createElement("h3", null, nombre),
    React.createElement("p", null, "$" + precio + " COP"),
    React.createElement("button", { onClick: () => pedir(nombre) }, "Ordenar")
  );
};
```

3. ¿Qué son Clases en React y su Evolución con Hooks?

En las versiones clásicas de React (anteriores a React 16.8), las **Clases** eran el único mecanismo disponible para declarar componentes con estado interno (*state*) y acceso a los métodos del ciclo de vida. Se creaban heredando de la clase base React.Component e implementando obligatoriamente el método render():

```
import React, { Component } from 'react';

class ContadorPizzas extends Component {
  constructor(props) {
    super(props);
    this.state = { cantidad: 1 };
    this.incrementar = this.incrementar.bind(this); // Enlace manual del 'this'
  }

  componentDidMount() {
    console.log("Componente montado en el DOM real");
  }

  componentWillUnmount() {
    console.log("Limpieza de recursos antes de desmontar");
  }

  incrementar() {
    this.setState(prev => ({ cantidad: prev.cantidad + 1 }));
  }

  render() {
    return (
      <div>
        <p>Pizzas: {this.state.cantidad}</p>
        <button onClick={this.incrementar}>+ Añadir</button>
      </div>
    );
  }
}
```

La Revolución de los Componentes Funcionales con Hooks (React 16.8+)

La introducción de los **Hooks** transformó por completo la industria. Al permitir que funciones simples manejen estado local mediante useState y efectos colaterales con useEffect, se eliminó la necesidad de clases, resolviendo los problemas históricos asociados al contexto léxico del this y reduciendo notablemente las líneas de código:

```
import React, { useState, useEffect } from 'react';

const ContadorPizzasModerno = () => {
  const [cantidad, setCantidad] = useState(1);

  useEffect(() => {
    console.log("Efecto montado / actualizado");
    return () => console.log("Limpieza automática al desmontar");
  }, [cantidad]);

  return (
    <div>
      <p>Pizzas: {cantidad}</p>
      <button onClick={() => setCantidad(c => c + 1)}>+ Añadir</button>
```

```
    </div>
  );
};
```

Criterio	Componentes de Clase (`React.Component`)	Componentes Funcionales Modernos con Hooks
Sintaxis	Clases ES6 con herencia y método render().	Funciones puras de JavaScript (arrow functions).
Estado	this.state y this.setState().	Hook useState() desestructurado.
Ciclo de Vida	Métodos dispersos (componentDidMount, Update, etc.).	Hook unificado y declarativo useEffect().
Manejo de `this`	Requiere binding explícito en el constructor.	Ámbito léxico nativo sin problemas de this.
Estado Actual	Código legado / en mantenimiento.	**Estándar oficial y recomendado por React**.

4. Principales Eventos en React y Sistema SyntheticEvent

Para garantizar máxima portabilidad entre navegadores y optimizar el consumo de memoria, React no asocia directamente oyentes en cada nodo del DOM real. En su lugar, utiliza un sistema centralizado de delegación de eventos basado en el objeto `**SyntheticEvent**`.

SyntheticEvent es un contenedor que envuelve el evento nativo del navegador, proporcionando una interfaz idéntica en Chrome, Firefox, Safari o navegadores móviles de acuerdo con la norma W3C.

> **Diferencias Clave frente a Eventos Nativos del DOM:**
>
> - **Nomenclatura camelCase:** onClick en vez de onclick; onSubmit en vez de onsubmit.
> - **Paso de Funciones:** Se pasa una referencia a la función onClick={manejador} y no una cadena de texto.
> - **Prevención de Comportamiento por Defecto:** Se debe invocar obligatoriamente e.preventDefault(); retornar false no surte efecto.

Catálogo de Eventos Principales en React:

Categoría	Evento React	Momento de Disparo	Aplicación en Pizzería Bella Italia
Mouse (Ratón)	onClick	Clic primario sobre un elemento.	Botón para seleccionar pizza o confirmar comanda.
	onMouseEnter / Leave	Cursor entra o sale del área.	Efecto visual de elevación en la tarjeta de pizza.
Formularios	onChange	Cambio en el valor del input/select.	Componentes controlados para capturar teléfono y dirección.
	onSubmit	Envío del formulario.	Validación previa al llamado de la API REST Spring Boot.
	onFocus / onBlur	Elemento gana o pierde el foco.	Validar que el teléfono tenga 10 dígitos al salir del input.
Teclado	onKeyDown	Presión de una tecla.	Cerrar ventana modal con tecla Escape.
Táctil (Mobile)	onTouchStart / End	Interacción en pantallas táctiles.	Navegación gestual por el catálogo en smartphones.

Ejemplo: Componente Controlado con `onChange` y `onSubmit`:

```
const FormularioCliente = () => {
  const [cliente, setCliente] = useState({ nombre: '', telefono: '' });

  const handleChange = (e) => {
    // e es un SyntheticEvent normalizado
    setCliente({ ...cliente, [e.target.name]: e.target.value });
```

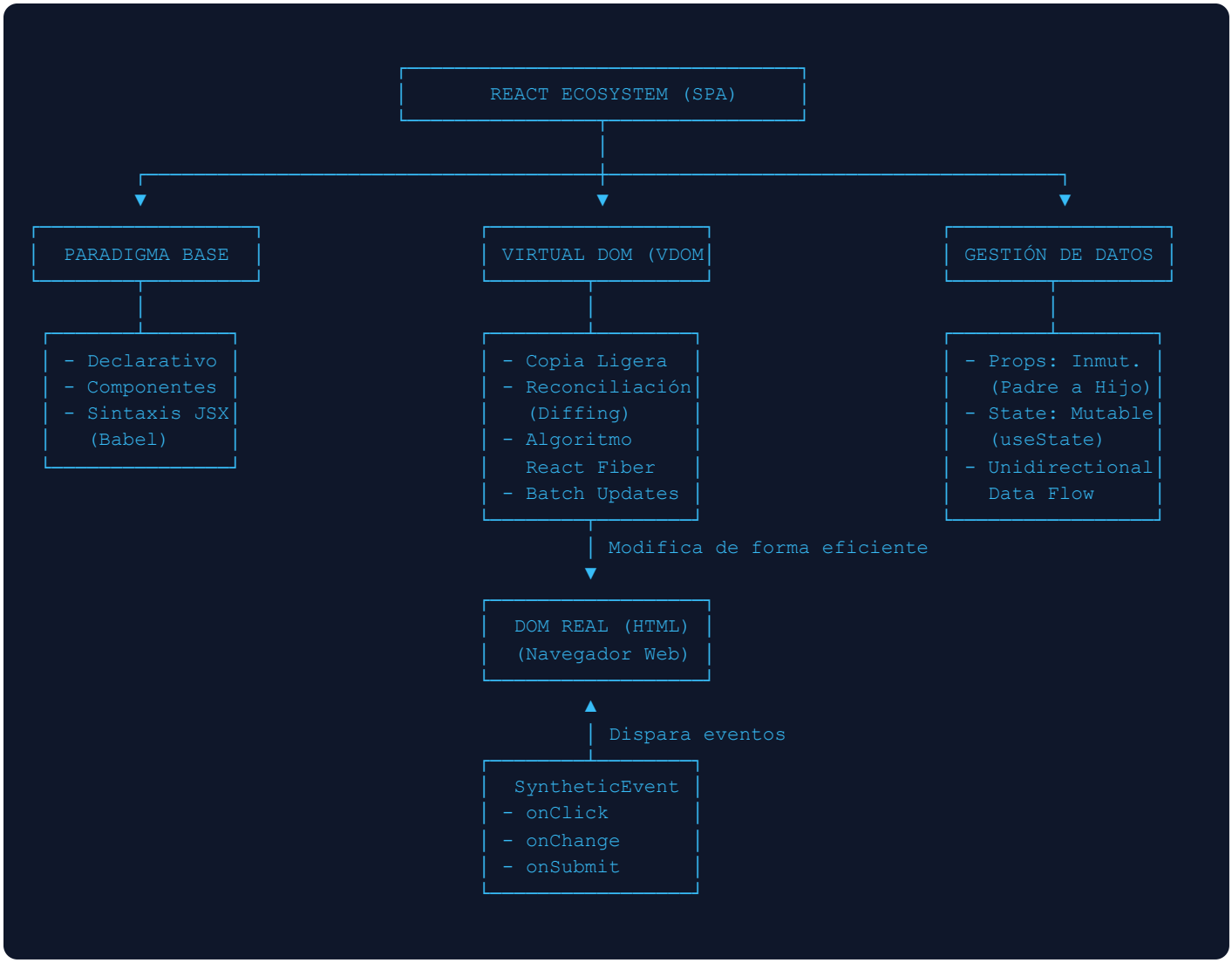
```
  };

  const handleSubmit = (e) => {
    e.preventDefault(); // Evita recarga del navegador
    console.log("Enviando cliente a la API Spring Boot:", cliente);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="nombre" value={cliente.nombre} onChange={handleChange} required />
      <input name="telefono" value={cliente.telefono} onChange={handleChange} required />
      <button type="submit">Registrar Cliente</button>
    </form>
  );
};
```

5. Mapa Conceptual de la Arquitectura de React

La siguiente estructura conceptual representa de forma sinóptica las interrelaciones fundamentales del ecosistema React:



Pilares Arquitectónicos Explicados:

1. **Virtual DOM y Algoritmo de Reconciliación (Fiber):** React conserva una copia virtual del árbol DOM en memoria. Ante cambios de estado, ejecuta un proceso de comparación (*Diffing*) y aplica exclusivamente las modificaciones necesarias al DOM real por lotes (*batching*), maximizando la fluidez de renderizado a 60 FPS.
2. **Flujo Unidireccional de Datos (*One-Way Data Flow*):** Las propiedades (*Props*) viajan exclusivamente de arriba hacia abajo (de componentes padres a hijos), mientras que los eventos y funciones callbacks permiten notificar cambios hacia arriba, garantizando alta predictibilidad en el estado.
3. **Componentización Modular:** Cada elemento de la pantalla es una función autónoma reutilizable que combina su lógica, marcado y estilos en una sola unidad.

6. Matriz de Cumplimiento de la Rúbrica SENA (IE-GA7-220501096-AA4-EV01)

Criterio Evaluado en la Guía 7	Resultado	Evidencia Aportada en el Documento
1. El documento sigue las normas de presentación (portada, introducción, objetivo).	CUMPLE	Informe estructurado formal de 8 páginas con normas SENA y portada institucional.
2. Presenta la diferencia conceptual y técnica entre React y JSX.	CUMPLE	Sección 2: Rol de biblioteca vs sintaxis, Babel y comparación de código.
3. Explica qué son las clases en React, su ciclo de vida y contraste con Hooks.	CUMPLE	Sección 3: React.Component, fases de ciclo de vida y migración a Hooks.
4. Detalla los principales eventos de React y el sistema SyntheticEvent.	CUMPLE	Sección 4: Catálogo de eventos, camelCase y ejemplos de inputs controlados.
5. Incluye un mapa conceptual completo y representativo de React.	CUMPLE	Sección 5: Diagrama arquitectónico del Virtual DOM, Fiber, Props y State.
6. Describe las principales características del framework/biblioteca.	CUMPLE	Sección 5 y 6: Component-based, declarativo, reconciliación y ecosistema.

7. Conclusiones

1. React y JSX forman un ecosistema simbiótico donde la biblioteca aporta la reactividad y reconciliación eficiente del Virtual DOM, mientras que JSX otorga una sintaxis declarativa intuitiva que compila a JavaScript puro.
2. La adopción masiva de Componentes Funcionales con Hooks reemplazó con éxito a las clases de React, eliminando el boilerplate y las complejidades del contexto léxico del this.
3. El sistema de eventos sintéticos SyntheticEvent unifica el comportamiento de eventos en todos los navegadores web modernos, facilitando la construcción de aplicaciones robustas como el frontend de la Pizzería Bella Italia.